

Computer Organization & Assembly Language

Lab 08

Lab Instructor: Mr. Tariq Mehmood Butt

Topic: Procedures & Stack Implementation

Instructions:

Pass all parameters through **AX** and **CX** registers.

Each procedure must return its result in **AX**.

Ensure that all registers used inside procedures are preserved using **PUSH** and **POP**.

LAB TASKS

Write a **main** procedure that should:

- Call each procedure in sequence.
- Display the results returned by each procedure appropriately (*no need for those procedures that already display results*).
- Demonstrate parameter passing through **AX** and **CX** registers.

Task 1:

Write a procedure that displays the **binary representation** of a 16-bit number passed in **AX**.

Task 2:

Write a procedure that displays the **content of a register in hexadecimal**.

Task 3:

Write a procedure that accepts a number in a register and prints whether it is:

- Even or Odd
- Positive or Negative

Task 4:

Write a procedure that accepts an **ASCII character in AL** and prints whether it is:

- Capital Letter
- Small Letter
- Digit
- Other Character

Task 5:

Write a procedure that performs **multiplication using repeated addition**.
The number is in **AX**, and the multiplier is in **CX**.
Store and return the result in **AX**.

HOME TASKS

Task 1:

Write a procedure that **prints a string in reverse order**.
The **offset address of the string** is passed in **AX**, and the string terminates with a '\$' sign.

Task 2:

Write a procedure that **returns the maximum element of an array**.
The **base address of the array** is passed in **AX**, and the **array length** is in **CX**.
Return the maximum value in **AX**.

Task 3:

Write a procedure that checks whether a \$-terminated string is a **palindrome**.
Return 1 in AX if palindrome, 0 otherwise.

Task 4:

Write a procedure that counts and returns the number of 1-bits in the binary representation of a 16-bit number passed in AX.

Task 5:

Write a procedure that counts the number of words in a \$-terminated string.
A word is defined as a sequence of non-space characters separated by one or more spaces. The offset address of the string is passed in **AX**, and the procedure should return the total word count in **AX**.

Write a main procedure to show the working of all these procedures.